

喵喵乐消消---棋盘渲染系统改造开发文档

坐标系重构 · 边框颜色系统 · 异形棋盘分层

实际牌尺寸 60px · 顶点压中心坐标系

0 背景与改造目标

现状问题

- ① 坐标系混乱：card.row / card.col 在不同地方既当格子编号又当像素用，层偏移叠加在随机坐标上，棋盘整体散乱。
- ② 全局缩放吃掉牌尺寸：设计稿 472px 棋盘被缩放到 326px 可用宽，实际牌渲染尺寸只有 ~37.5px，比设计意图小 22%。
- ③ 边框颜色随机：自定义边框颜色未体现任何层次语义，玩家无法快速判断哪张牌可以点击。
- ④ 异形棋盘无约束：高层牌扩散时不检查形状边界，破坏异形棋盘的轮廓。

改造后目标

牌实际渲染尺寸：60px（从 37.5px 提升 60%，可点击面积 $\times 2.56$ ）

坐标系：绝对像素坐标（顶点压中心，层越高区域越向内收缩，永远不溢出屏幕）

边框颜色：5 级冷暖渐变，颜色直接表达层次，玩家无需思考

异形棋盘：上层牌候选位置过滤 isInShape()，形状轮廓不被破坏

生成逻辑不变：reverseGenerate()、isSolvable()、generateBoard() 主体逻辑保持原样

1 核心参数

以下常量统一在 boardConstants.js 中定义，全局引用，不在各处硬编码。

```
// — boardConstants.js —
```

```
// 牌尺寸（实际屏幕像素，无缩放）
export const TW = 60;           // 牌宽 px
export const TH = 60;           // 牌高 px
export const HW = TW / 2;       // 半格宽 = 30px（层间偏移单位）
export const HH = TH / 2;       // 半格高 = 30px

// 棋盘布局
export const GRID_COLS = 8;     // 底层列数
export const GRID_ROWS = 6;     // 底层行数
export const BOARD_MARGIN = 32; // 屏幕安全边距 px

// 底层棋盘实际占用尺寸（供 canvas 或容器设置）
// 宽 = (GRID_COLS-1) × HW + TW = 7×30 + 60 = 270px
// 高 = (GRID_ROWS-1) × HH + TH = 5×30 + 60 = 210px
export const BOARD_W = (GRID_COLS - 1) * HW + TW; // 270px
export const BOARD_H = (GRID_ROWS - 1) * HH + TH; // 210px

// 边框颜色（按 layer 索引）
export const LAYER_BORDER = [
  '#4ECDC4', // Layer 0 - 青绿，可点击
  '#7BC67E', // Layer 1 - 黄绿，轻度压盖
  '#F7B731', // Layer 2 - 琥珀，中度压盖
  '#F0813A', // Layer 3 - 橙色，深度压盖
  '#E05252', // Layer 4+ - 红色，最深锁定
];

// 亮度/饱和度（配合边框颜色的视觉衰减）
export const LAYER_FILTER = [
  'none',
  'brightness(0.93)',
  'brightness(0.84)',
  'brightness(0.73) saturate(0.65)',
  'brightness(0.60) saturate(0.45)',
];
```

参数验算

参数	计算过程	结果	状态
可用宽	390 - 32×2	326px	充裕
底层棋盘宽	(8-1)×30 + 60	270px	270 < 326
宽度余量	326 - 270	56px	宽松
底层棋盘高	(6-1)×30 + 60	210px	210px
高度余量（去顶底栏）	540 - 210	330px	非常宽裕
层间偏移溢出	层越高→区域向内收缩	0px	永远不溢出

2 坐标系设计（顶点压中心）

2.1 核心原理

羊了个羊的整齐感来源于一个固定规则：上层牌的左上角顶点，落在下层牌的中心点。每张下层牌最多可被 4 张上层牌压住（右下、左下、右上、左上各一张）。

关键性质：层越高，有效区域越向内收缩

Layer 0（底层）：占满整个 270×210px 区域

Layer 1：四周各收缩 HW=30px，有效区变小

Layer N：比底层小 N×30px 一圈，始终在底层边界框内

→ 无论叠多少层，上层牌永远不超出底层棋盘边界，不需要预留额外空间

2.2 坐标字段定义

改造后每张牌使用 `card.x / card.y` 存储绝对像素坐标（左上角），不再使用 `card.row / card.col` 作为像素值。

字段	类型	含义	改造前	改造后
<code>card.x</code>	number	牌左上角 X 像素（相对棋盘容器）	<code>card.col</code> （混用）	绝对像素 X
<code>card.y</code>	number	牌左上角 Y 像素（相对棋盘容器）	<code>card.row</code> （混用）	绝对像素 Y
<code>card.layer</code>	number	层编号，0=底层可点击	不变	不变
<code>card.type</code>	string	牌面类型	不变	不变
<code>card.zone</code>	string	区域标识	不变	不变
<code>card.id</code>	number	唯一 ID	不变	不变

2.3 坐标生成规则

底层坐标（Layer 0）

底层牌在半格对齐的网格上均匀分布，步长为 HW/HH，保证相邻格子之间的间距一致。

```

// genBaseCoords() - 生成底层合法坐标集合
// 输出的坐标即为棋盘容器内的绝对像素位置

function genBaseCoords(boardType) {
  const coords = [];
  for (let row = 0; row < GRID_ROWS; row++) {
    for (let col = 0; col < GRID_COLS; col++) {
      const x = BOARD_MARGIN + col * HW;
      const y = BOARD_MARGIN + row * HH;
      // 异形棋盘: 过滤掉不在形状内的坐标
      if (isInShape(boardType, x, y)) {
        coords.push({ x, y });
      }
    }
  }
  return coords; // 每个点都是合法的底层放牌位置
}

// 标准棋盘 (boardType = "standard") 时 isInShape 始终返回 true
// 底层共 8×6 = 48 个格子, 步长 30px
// x 范围: [32, 32+7×30] = [32, 242], 加牌宽 60 = 302px < 326px

```

上层坐标 (Layer 1+)

每张已有牌提供 4 个候选上层位置（四个角落方向各一个），候选位置必须同时满足 3 个条件才被采用。

```

// getValidUpperPositions() - 获取一张牌的合法上层候选位置

function getValidUpperPositions(card, boardType, boardW, boardH) {
  const candidates = [
    { x: card.x + HW, y: card.y + HH }, // 右下
    { x: card.x - HW, y: card.y + HH }, // 左下
    { x: card.x + HW, y: card.y - HH }, // 右上
    { x: card.x - HW, y: card.y - HH }, // 左上
  ];

  return candidates.filter(p =>
    isInShape(boardType, p.x, p.y) && // ① 在形状集合内
    p.x >= BOARD_MARGIN && // ② 左边界
    p.x + TW <= boardW - BOARD_MARGIN && // ② 右边界
    p.y >= BOARD_MARGIN && // ③ 上边界
    p.y + TH <= boardH - BOARD_MARGIN // ③ 下边界
  );
}

// 注意: 标准棋盘中, isInShape 返回 true, 边界检查是唯一过滤条件
// 异形棋盘中, isInShape 额外过滤掉形状轮廓外的候选位置

```

2.4 遮盖关系判断

改造后的遮盖判断基于绝对像素坐标，**纯整数运算，无浮点误差**。阈值是 TW（整格），不是 HW（半格）。

```
// isCoveredBy(lower, upper) - 判断 upper 是否压住 lower
// 条件: upper 在 lower 的正上方一层, 且两牌在视觉上有交叠

function isCoveredBy(lower, upper) {
  if (upper.layer !== lower.layer + 1) return false;

  const dx = Math.abs(upper.x - lower.x);
  const dy = Math.abs(upper.y - lower.y);

  // 两牌左上角坐标差 < TW(60px) 时视觉上有交叠
  // 原理: 上层牌角落恰好压在下层牌中心(HW=30px 处),
  // 因此差值 < TW 即为遮盖, 而非 < HW
  return dx < TW && dy < TH;
}

// 获取场上所有当前可点击的牌
function getClickable(cards) {
  return cards.filter(card =>
    !cards.some(other => isCoveredBy(card, other))
  );
}
```

与旧版 isCovered() 的区别

旧版: $\text{Math.abs}(\text{other.row} - \text{card.row}) < 1$ ← 单位混用, 有浮点误差

新版: $\text{Math.abs}(\text{upper.x} - \text{lower.x}) < \text{TW}$ ← 纯像素整数, 精确

同步更新 isSolvable() 内部调用: 将遮盖判断换成新函数即可, 外部接口不变

3 边框颜色系统

3.1 设计原则

边框颜色是玩家判断层次的主要视觉信号。颜色从冷（青绿）到暖（红色），对应从「可立即点击」到「深度锁定」，与人类本能认知（绿灯走/红灯停）对齐，无需学习。

Layer 0 #4ECDC4 可点击	Layer 1 #7BC67E 轻度压盖	Layer 2 #F7B731 中度压盖	Layer 3 #F0813A 深度压盖	Layer 4+ #E05252 最深锁定

3.2 完整规范表

Layer	边框色	Hex	CSS filter	底色	语义
0	■ 青绿	#4ECDC4	none	#FFFFFF	可立即点击，脉冲动画
1	■ 黄绿	#7BC67E	brightness(0.93)	#F4FFF0	再消一张即可触达
2	■ 琥珀	#F7B731	brightness(0.84)	#FFFBF0	还差 2 步
3	■ 橙色	#F0813A	brightness(0.73) saturate(0.65)	#FFF5EE	需要规划路径
4+	■ 红色	#E05252	brightness(0.60) saturate(0.45)	#FFF0F0	当前锁定，终极目标区

3.3 谜感保留原则

颜色只告诉层次，不告诉数量

揭示：这张牌有多深（层次） → 帮助玩家制定策略

不揭示：上面压了几张牌 → 保留谜感

不揭示：压盖牌的类型 → 核心谜题

设计意图：红色边框告诉玩家「这里很难」，但不告诉「有多少张」——这个张力是游戏乐趣的来源。

因此不实现叠牌数量徽章（×3、×4 标记）。

3.4 渲染实现

```
// renderCard(card, tileElement) - 渲染单张牌
// 在现有渲染函数中，找到赋值 borderColor 的位置，替换为以下逻辑

function renderCard(card, el) {
  const layerIdx = Math.min(card.layer, 4);

  // 位置（直接用绝对像素，不需要换算）
  el.style.left = card.x + "px";
  el.style.top = card.y + "px";
  el.style.width = TW + "px";
  el.style.height = TH + "px";

  // 层次（layer 越小 zIndex 越高，顶层牌在最前面）
  el.style.zIndex = (10 - card.layer) * 10;

  // 边框颜色 ← 替换原有的随机颜色逻辑
  el.style.borderColor = LAYER_BORDER[layerIdx];

  // 亮度衰减
  el.style.filter = LAYER_FILTER[layerIdx];

  // Layer 0 加脉冲动画，提示可点击
  if (card.layer === 0) {
    el.classList.add("tile-clickable");
  } else {
    el.classList.remove("tile-clickable");
  }
}

/* CSS - tile-clickable 脉冲动画 */
@keyframes border-pulse {
  0%, 100% { box-shadow: 0 0 0 0 rgba(78, 205, 196, 0); }
  50%      { box-shadow: 0 0 0 6px rgba(78, 205, 196, 0.25); }
}
.tile-clickable {
  animation: border-pulse 2s infinite;
}
```


4 异形棋盘分层

4.1 核心机制

异形棋盘（爱心棋盘、菱形棋盘、漏斗棋盘、蝴蝶棋盘、双域棋盘、花瓣棋盘等）通过 `isInShape(boardType, x, y)` 函数约束上层牌的生成范围。只需在 `getValidUpperPositions()` 的 `filter` 里加一行判断，标准棋盘和异形棋盘使用完全相同的分层逻辑，无需分支处理。

自然产生的纵深效果（无需额外控制）

形状边缘的牌：4 个候选角落中有些在形状外被过滤，可用角落少 → 上层牌少 → 层数浅

形状中心的牌：4 个候选角落都在形状内 → 上层牌多 → 层数深

结果：中心自然隆起（难），边缘自然平坦（易）——与 `buryRate` 参数天然配合，无需额外代码

4.2 `isInShape()` 实现

```
// isInShape(boardType, x, y)
// 判断坐标 (x, y) 是否在指定形状内
// x, y 为棋盘容器内的绝对像素坐标（牌的左上角）

function isInShape(boardType, x, y) {
  // 标准矩形棋盘：始终返回 true
  if (boardType === "standard") return true;

  // 对齐到半格网格，避免浮点误差
  const sx = Math.round(x / HW) * HW;
  const sy = Math.round(y / HH) * HH;

  // 查形状集合 (O(1))
  return getShapeSet(boardType).has(`${sx}_${sy}`);
}

// 懒加载：每种 boardType 的 Set 只生成一次
const _shapeSets = {};
function getShapeSet(boardType) {
  if (!_shapeSets[boardType]) {
    _shapeSets[boardType] = buildShapeSet(boardType);
  }
  return _shapeSets[boardType];
}
```

三类形状的坐标集生成

```

// buildShapeSet(boardType) - 生成形状坐标集
// 返回 Set<string>, 每个元素为 "x_y" 格式

function buildShapeSet(boardType) {
  const pts = buildShapeCoords(boardType);
  return new Set(pts.map(p => `${p.x}_${p.y}`));
}

// — 菱形: 曼哈顿距离 ≤ 半径 —————
function buildDiamond(cx, cy, R) {
  const pts = [];
  for (let y = cy - R; y <= cy + R; y += HH)
    for (let x = cx - R; x <= cx + R; x += HW)
      if (Math.abs(x - cx) / R + Math.abs(y - cy) / R <= 1)
        pts.push({ x, y });
  return pts;
}

// — 圆形: 欧氏距离 ≤ 半径 —————
function buildCircle(cx, cy, R) {
  const pts = [];
  for (let y = cy - R; y <= cy + R; y += HH)
    for (let x = cx - R; x <= cx + R; x += HW)
      if ((x - cx) ** 2 + (y - cy) ** 2 <= R ** 2)
        pts.push({ x, y });
  return pts;
}

// — 不规则形 (心形、蝴蝶等) —————
// 直接使用现有的 LOVE_BOARD_COORDS、BUTTERFLY_BOARD_COORDS 等
// 只需确保坐标值已对齐到 HW/HH 的倍数 (像素坐标, 非格子编号)
// function buildHeart() { return HEART_BOARD_COORDS; }

```

4.3 现有坐标数组迁移

你现有的 LOVE_BOARD_COORDS、DIAMOND_BOARD_COORDS 等数组, 若存储的是格子编号 (row/col), 需要换算为像素坐标。若已是像素坐标, 确保对齐到 HW/HH 倍数即可。

```

// 若现有坐标是格子编号格式 { row, col }, 一次性转换:
const LOVE_BOARD_COORDS_PX = LOVE_BOARD_COORDS.map(({ row, col }) => ({
  x: BOARD_MARGIN + col * HW,
  y: BOARD_MARGIN + row * HH,
}));

// 若已是像素格式, 对齐到半格:
const aligned = DIAMOND_BOARD_COORDS.map(({ x, y }) => ({
  x: Math.round(x / HW) * HW,
  y: Math.round(y / HH) * HH,
}));

```

```
// 完成后，在 buildShapeSet() 里直接引用像素坐标数组
```

5 与现有生成算法对接

5.1 改动范围总览

原则：最小改动，最大收益

generateBoard()、isSolvable()、addInterferenceCards()、shuffleArr() — 主体逻辑不变

reverseGenerate() — 仅修改坐标字段名 (row→y, col→x) 和上层位置生成方式

isCovered() — 遮盖判断改为像素版本 (约 5 行)

renderCard() — 直接用 card.x/card.y，删掉换算代码，加边框颜色逻辑

BOARD_COORDS 等坐标数组 — 换算为像素坐标格式

5.2 reverseGenerate() 对接

生成逻辑的核心不变，只改坐标字段的使用方式：

```
// reverseGenerate() 改动点 (伪代码，只展示需要修改的部分)

// ① 底层坐标来源：改用 genBaseCoords()
// 原来：从 BOARD_COORDS 取 { row, col }
// 现在：从 genBaseCoords(boardType) 取 { x, y }
const baseCoords = genBaseCoords(boardType); // 输出绝对像素坐标

// ② 查找合法上层位置：改用 getValidUpperPositions()
// 原来：findPlaceablePositions() 基于格子坐标
// 现在：getValidUpperPositions(card, boardType, boardW, boardH)
const upperCandidates = getValidUpperPositions(card, boardType, boardW, boardH);

// ③ 放回牌时，坐标字段名变化：
// 原来：{ row, col, layer, type, zone }
// 现在：{ x, y, layer, type, zone }
placed.push({ x: pos.x, y: pos.y, layer: newLayer, type, zone });

// ④ 其余逻辑 (typePool 构建、深埋策略、noiseRate、难度控制) 完全不变
```

5.3 isSolvable() 对接

只需将内部的遮盖判断替换为新版 isCoveredBy()，外部接口 (参数、返回值) 完全不变。

```
// isSolvable() 内部修改（仅遮盖判断部分）

// 原来（格子坐标）：
// const rowOverlap = Math.abs(other.row - card.row) < 1;
// const colOverlap = Math.abs(other.col - card.col) < 1;
// return rowOverlap && colOverlap;

// 现在（像素坐标）：
function _isCoveredInSolvable(lower, upper) {
  if (upper.layer !== lower.layer + 1) return false;
  return Math.abs(upper.x - lower.x) < TW &&
    Math.abs(upper.y - lower.y) < TH;
}

// 其余 DFS 回溯逻辑、深度限制、候选数限制 - 全部不变
```

5.4 generateBoard() 对接

主函数不需要改动逻辑，只需删掉全局缩放相关的代码。

```
// generateBoard() 改动点

// ① 删掉全局缩放计算
// 原来：const scale = Math.min((100vw-24px)/417, (100vh-24px)/876)
// 现在：不需要 scale，棋盘尺寸固定为 BOARD_W × BOARD_H

// ② 棋盘容器尺寸改为常量
// 原来：canvas 472 × 352px（含旧方案层偏移预留）
// 现在：容器设为 BOARD_W × BOARD_H = 270 × 210px（底层尺寸）
// 上层牌超出容器边界是正常的（视觉效果），容器用 overflow: visible

// ③ 层样式映射（buildLayerOffsetMap）仍然保留，用于控制 z-index
// zIndex 计算改为：(10 - layer) * 10

// ④ 其余逻辑（层配置、牌型分配、干扰牌、兜底验证）完全不变
```

6 改动文件清单

6.1 需要修改的文件

文件 / 模块	改动内容	改动量	优先级
boardConstants.js (新建)	统一定义 TW/TH/HW/HH/BOARD_W/BOARD_H/LAYER_BORDER/LAYER_FILTER	新建 ~40 行	P0
renderCard()	① 用 card.x/card.y 替代 row/col ② 替换 borderColor 为 LAYER_BORDER[layer] ③ 加 LAYER_FILTER[layer] ④ Layer 0 加/去 tile-clickable class	~10 行	P0
CSS 样式文件	新增 .tile-clickable 脉冲动画；删除全局 scale transform	~15 行	P0
isCovered() isCoveredBy()	/ 遮盖判断改为像素版 dx <TW && dy <TH	~8 行	P1
reverseGenerate()	坐标字段 row→y, col→x；底层坐标改用 genBaseCoords()；上层位置改用 getValidUpperPositions()	~20 行	P1
isSolvable()	内部遮盖判断同步为像素版	~5 行	P1
generateBoard()	删除全局缩放；棋盘容器改为 BOARD_W×BOARD_H	~8 行	P1
BOARD_COORDS 等 坐标数组	格子编号坐标换算为像素坐标（或在 buildShapeSet 里换算）	~10 行 / 文件	P2
isInShape() getShapeSet() (新建)	/ 形状约束函数，含懒加载 Set 缓存	新建 ~40 行	P2
genBaseCoords()	生成底层合法坐标集（含异形过滤）	新建 ~20 行	P1
getValidUpperPositions()	四方向候选位置 + 形状约束 + 边界过滤	新建 ~20 行	P1

6.2 不需要修改的文件

文件 / 函数	原因
getLevelConfig()	关卡配置与坐标系无关
getLayerConfig()	层配置与坐标系无关
buildTypePool()	牌型分配与坐标系无关

文件 / 函数	原因
addInterferenceCards()	干扰牌添加逻辑不依赖坐标系（但其输出的坐标字段名需与新格式一致）
shuffleArr()	工具函数，无关
getDifficultyConfig()	难度配置与坐标系无关
buildLayerOffsetMap()	只用于 zIndex 控制，改为常量公式后可选保留

7 接入验收标准

7.1 视觉验收

验收项	预期结果	验证方式
牌实际渲染尺寸	60px，无缩放	用 DevTools 测量 DOM 元素尺寸
棋盘不溢出屏幕	所有牌（含高层）在 326px 可用宽内	真机或 390px 视口检查
Layer 0 边框	青绿色 #4ECDC4，有脉冲动画	目测 + DevTools computed style
Layer 1-4 边框	依次黄绿/琥珀/橙色/红色，亮度递减	目测多层叠牌区域
标准棋盘整齐感	所有牌对齐半格网格，视觉整齐	目测，无随机散落感
异形棋盘轮廓	高层牌不超出形状边界	对比底层形状与高层分布

7.2 功能验收

验收项	预期结果	验证方式
isSolvable() 通过率	逆向生成的布局 100% 通过	console.warn 无输出
点击判断正确	只有无遮盖的牌响应点击	点击被压牌，无响应
异形棋盘可解	isSolvable 通过，形状内有完整消除路径	各 boardType 运行测试
小屏适配	360px 屏幕棋盘不溢出	DevTools 模拟 360px
多局不同	同一关卡每局布局视觉不同	连续进入同一关 5 次对比

7.3 小屏兜底

```
// 对屏幕宽 < 360px 的设备，动态计算 TW
// 在应用初始化时调用一次

function initTileSize() {
  const availW = window.innerWidth - BOARD_MARGIN * 2;
  // 底层棋盘宽 = TW * (GRID_COLS - 1) / 2 + TW = TW * 4.5
  // TW_max = availW / 4.5
  const twMax = Math.floor(availW / 4.5 / 4) * 4; // 对齐到 4px
```



```
    return Math.min(60, twMax); // 不超过设计值 60px
}

// 390px 屏幕: min(60, floor(326/4.5/4)*4) = min(60, 72) = 60px
// 360px 屏幕: min(60, floor(296/4.5/4)*4) = min(60, 64) = 60px
// 320px 屏幕: min(60, floor(256/4.5/4)*4) = min(60, 56) = 56px (自动降级)
```

8 快速参考

8.1 关键公式速查

```
// —— 坐标换算 ——  
底层牌坐标: x = BOARD_MARGIN + col * HW  
              y = BOARD_MARGIN + row * HH  
  
上层牌坐标（四个方向）：  
  右下: { x: card.x + HW, y: card.y + HH }  
  左下: { x: card.x - HW, y: card.y + HH }  
  右上: { x: card.x + HW, y: card.y - HH }  
  左上: { x: card.x - HW, y: card.y - HH }  
  
// —— 遮盖判断 ——  
isCoveredBy(lower, upper):  
  upper.layer === lower.layer + 1  
  && Math.abs(upper.x - lower.x) < TW    // < 60px  
  && Math.abs(upper.y - lower.y) < TH    // < 60px  
  
// —— 棋盘尺寸 ——  
BOARD_W = (GRID_COLS - 1) * HW + TW    // (8-1)*30+60 = 270px  
BOARD_H = (GRID_ROWS - 1) * HH + TH    // (6-1)*30+60 = 210px  
  
// —— 边框颜色 ——  
LAYER_BORDER[Math.min(card.layer, 4)]  
  → 0: #4ECD C4  1: #7BC6 7E  2: #F7B7 31  3: #F081 3A  4: #E052 52  
  
// —— 最大牌尺寸（小屏兜底） ——  
TW_max = Math.floor(availW / 4.5 / 4) * 4  
390px 屏: 72px(上限) → 取设计值 60px  
320px 屏: 56px      → 自动降级为 56px
```

8.2 术语对照表

术语	含义	单位
TW / TH	牌宽/高 (Tile Width/Height)	px, 固定 60
HW / HH	半格宽/高 (Half Width/Height) = TW÷2	px, 固定 30
card.x / card.y	牌左上角绝对像素坐标 (相对棋盘容器)	px
card.layer	层编号, 0 = 底层/可点击, 数字越大越深	整数
BOARD_W BOARD_H	/ 底层棋盘实际占用尺寸 (270×210px)	px, 固定

术语	含义	单位
BOARD_MARGIN	棋盘容器内边距（32px）	px
isInShape()	判断坐标是否在当前 boardType 的形状内	boolean
shapeSet	形状有效坐标的 Set<"x_y" string>，O(1) 查找	Set
isCoveredBy()	判断上层牌是否压住下层牌（像素坐标版）	boolean
LAYER_BORDER	5 级边框颜色数组，按 layer 索引	string[]